Closed-Loop LLM Agents for Multi-UAV Wildfire Rescue

Author One
Department, Institution, City, Postal Code, Country

Author Two
Department, Institution, City, Postal Code, Country

August 18, 2026

Abstract

Multi-UAV wildfire rescue requires uncertain aerial observations to be translated into fleet missions that satisfy resource and safety constraints and remain effective as operational conditions change. We investigate whether large language models (LLMs) can serve as the high-level closed-loop decision core of such a system.

We formulate wildfire rescue as a closed-loop decision problem. The resulting LLM-centered closed-loop decision framework integrates fire-situation understanding and resource-constrained mission planning with Validation, Memory, and Runtime Replanning. It is evaluated in a real-UAV-informed AirSim suite spanning four difficulty levels.

Across four downstream foundation-model configurations, the framework maintains high basic task completion, although execution quality declines as operational pressure increases. Controlled ablations show that Validation, Memory, and Runtime Replanning provide complementary benefits at different levels of operational pressure. On simple and normal episodes, a fine-tuned Qwen3-8B planner retains 98.24% of the Gemini 3.1 Pro teacher planner's average score while substantially reducing planning overhead. These results characterize the potential and limitations of LLMs for high-level closed-loop decision making in adaptive multi-UAV wildfire rescue. The implementation is available at `https://github.com/tongsuhdyem/Pipeline`.

Keywords: multi-UAV wildfire rescue; LLM agents; compact planner fine-tuning; high-level closed-loop decision making; real-to-sim evaluation

1 Introduction

Wildfires threaten ecosystems, infrastructure, and human safety (Bowman et al., 2020). UAVs can rapidly observe hazardous and inaccessible areas through flexible aerial sensing (Allan et al., 2022), but operational response requires more than aerial detection. Observations must be converted into task zones, assigned across a resource-limited fleet, and revised as the fire or UAV state changes. Multi-UAV wildfire rescue is therefore a closed-loop decision problem rather than an isolated perception or path-planning task.

The difficulty lies in satisfying two requirements at the same time. First, an initial mission must be feasible for the current fleet. Task priorities must be matched to vehicle locations, endurance, payload, communication, and safety limits (Floreano and Wood, 2015); otherwise, a semantically plausible plan may still be impossible to execute. Second, feasibility is temporary. Fire evolution, resource consumption, communication changes, and UAV faults can invalidate assignments that were valid when they were generated. A useful decision system must therefore reason not only about what the fleet should do now, but also about how the mission should be revised after the operational state changes. Initial resource-constrained mission planning and runtime replanning are coupled parts of the same decision problem.

Existing work has advanced fire detection, aerial scene understanding, damage assessment, navigation, and multi-UAV coordination (Erdelj et al., 2017). These capabilities provide essential parts of a rescue system, but they are commonly developed and evaluated as separate stages. As a result, strong perception does not by itself establish that detected fire conditions can be converted into a feasible fleet mission, while a feasible initial mission plan does not establish that the mission can remain effective under subsequent perturbations. The unresolved challenge is the high-level decision layer that connects fire-situation understanding, resource-constrained mission planning, execution feedback, and runtime replanning within one continuous loop.

Large language models (LLMs) and multimodal LLMs offer a flexible interface for this layer because they can connect visual interpretation, task decomposition, and high-level planning (Driess et al., 2023). However, free-form generation alone is insufficient for a resource-constrained, safety-relevant mission. An LLM may produce a coherent plan that violates fleet constraints, overlook a relevant lesson from a previous case, or continue to rely on an obsolete plan after execution conditions change. The central question of this work is therefore whether LLMs can serve as the high-level closed-loop decision core of a multi-UAV wildfire rescue system, and how Validation, Memory, and Runtime Replanning sustain that role as operational pressure increases.

We address this question by formulating multi-UAV wildfire rescue as a decision loop from aerial observation to fire-situation understanding, resource-constrained mission planning, Validation, execution feedback, and Runtime Replanning. The resulting LLM-centered closed-loop decision framework uses Vision Analysis and Mission Planning to transform observations into an initial fleet mission, but its design is organized around the risks that one-shot generation alone cannot address. Semantic Validation and Deterministic Rule Validation guard against plans that are infeasible or tactically implausible. Episodic and Lesson Memory retrieve prior cases and distilled guidance when the current mission-planning problem becomes less routine. Runtime Replanning responds to changes in the fire, fleet, resources, or schedule by repairing the active mission. Together, these mechanisms turn LLM planning from a one-shot output into a decision process that can be checked, informed by experience, and revised during execution.

To evaluate the resulting closed-loop capability, we construct a real-UAV-informed AirSim suite that progressively increases operational pressure. Observation, energy, payload, and communication conditions are calibrated from a real platform, while 80 fixed evaluation episodes are organized into four difficulty levels. The lower levels emphasize fire-situation understanding and resource-constrained mission planning; the higher levels introduce stronger operational pressure through tighter resource and scheduling conditions together with runtime perturbations. Six mission-level metrics separate task completion from response, mission time, event stability, resource pressure, and coordination, allowing

the evaluation to distinguish whether a system completes the rescue tasks from how well it preserves execution quality and mission continuity.

The main contributions of this work are summarized as follows:

1. We formulate resource-constrained multi-UAV wildfire rescue as a high-level closed-loop decision problem and develop an LLM-centered closed-loop decision framework that connects fire-situation understanding and resource-constrained mission planning with Validation, Memory, and Runtime Replanning.
2. We construct a real-UAV-informed AirSim evaluation suite that exposes the decision loop to controlled increases in operational pressure through resource limitations, scheduling constraints, and runtime perturbations across four difficulty levels, and measures both task completion and execution quality with six mission-level metrics.
3. We evaluate end-to-end behavior across multiple foundation-model configurations, characterize through controlled ablations how Validation, Memory, and Runtime Replanning contribute as operational pressure increases, and assess whether validated demonstrations can specialize a compact planner with lower planning overhead.

2 Related Work

2.1 UAV-Based Wildfire Rescue

UAVs provide a practical means of observing wildfires in hazardous and difficult-to-access areas, but operational wildfire rescue requires more than detecting fire in aerial imagery. It requires a decision chain that transforms observations into a current representation of the incident, allocates limited airborne resources to the resulting tasks, and updates the mission when the environment or fleet state changes. Two properties distinguish this problem from a collection of independent UAV capabilities. Decisions at one stage constrain the next stage, and decisions that are valid initially may cease to be valid during execution. Existing work has made substantial progress on the individual stages of this chain, including aerial perception, active situation monitoring, autonomous flight planning, and multi-UAV coordination, but the connection among these stages remains less developed.

At the perception stage, multimodal sensing improves the reliability of fire observations under the visually challenging conditions common in wildfire scenes. Thermal–RGB fusion has been used to detect fire and hotspots from UAV imagery, reducing sensitivity to smoke and illumination variation (Yuan et al., 2021). Transformer-based aerial semantic segmentation further supports the interpretation of burned regions, vegetation, and terrain (Zhang et al., 2022), while learning-based 3D reconstruction can provide geometric environmental models for navigation and inspection (Yang et al., 2022). These methods produce the visual and spatial evidence needed for rescue, but their outputs must still be converted into mission-level entities, such as fire points, task zones, and safety-relevant context, before they can drive rescue actions. This conversion is consequential: perception determines what the planner treats as a task, yet perception quality alone does not indicate whether the resulting tasks can be assigned to a resource-limited fleet or sustained throughout execution.

Situation awareness and planning research addresses part of this conversion. Autonomous exploration and mapping enable UAVs to update knowledge of unknown or changing disaster environments (Popovic et al., 2021); next-best-view planning selects

viewpoints that maximize information gain and reduces observation uncertainty (Bircher et al., 2016). On the action side, deep reinforcement learning has been applied to autonomous navigation in unknown environments (Xiao et al., 2021), and coverage path planning supports large-area search and perimeter scanning (Yu et al., 2022). However, these methods generally optimize sensing, navigation, or coverage as an individual capability. They do not by themselves produce a fleet-level, resource-constrained mission plan that explicitly links evolving fire information to task priorities, payload use, endurance, and execution status. In particular, optimizing where a UAV should observe or fly is different from deciding which combination of vehicles, routes, and payloads should serve multiple interdependent zones under a common mission objective.

Multi-UAV methods extend decision making from a single vehicle to a team. Multi-agent reinforcement learning has demonstrated cooperative navigation and task allocation capabilities (Yang et al., 2023), while the broader disaster-response literature highlights the value of UAV support and human–robot collaboration in search-and-rescue operations (Waharte and Trigoni, 2021). Nevertheless, wildfire rescue remains difficult because fire evolution, communication changes, energy depletion, and vehicle failures can invalidate an initially feasible allocation. The literature is therefore still dominated by modular or open-loop solutions: perception, mapping, path planning, and coordination are often evaluated separately, and task allocation is commonly assessed from a fixed initial state. This leaves a gap between initial coordination and mission continuity: a method may produce a valid allocation at one instant without specifying how the fleet should recover when the task set, vehicle state, or resource constraints change.

The methods reviewed above also assume different inputs, outputs, resource models, and evaluation protocols. Their reported results provide useful references for individual capabilities, but do not constitute a directly comparable end-to-end numerical baseline for a task that begins with aerial observations and continues through resource-constrained mission planning and runtime replanning. The relevant gap is therefore not the absence of perception, planning, or coordination algorithms in isolation. It is the absence of a common high-level decision loop that connects these capabilities and exposes how resource constraints and dynamic events propagate from fire-situation understanding to fleet execution.

2.2 LLM-Based Robotic Agents

Large language models (LLMs) provide an increasingly capable interface between high-level human intent and robot behavior. Rather than replacing low-level perception or control modules, they are commonly used to interpret language, decompose goals, select skills, and reason over structured observations. This role is particularly relevant to wildfire rescue, where a decision system must translate a changing fire situation and operational constraints into a coherent fleet mission rather than execute a fixed motion policy. For such a role, generative flexibility is only one requirement. The resulting decisions must also be grounded in available capabilities, checked against operational constraints, and revised when feedback invalidates their assumptions.

Language grounding and hierarchical planning are central directions in this area. SayCan combines language-model scoring with robot affordance estimates so that high-level instructions are grounded in executable skills (Ahn et al., 2022). Code as Policies uses language models to generate programmatic control policies, providing a flexible mechanism for composing robot behavior from functions and APIs (Liang et al., 2023). Related

design guidance has shown how ChatGPT can be used to generate robot workflows while exposing tool descriptions and environmental constraints to the model (Vemprala et al., 2023). Together, these approaches show that LLMs can support task decomposition and action sequencing, but they also make clear that generated actions require grounding and external feasibility checks. This distinction is central in multi-UAV missions: linguistic coherence does not guarantee that a set of assignments respects endurance, payload, communication, priority, and safety constraints simultaneously.

Embodied and multimodal models extend this reasoning interface to visual and physical state. RT-1 learns vision-language-action policies from large-scale real-robot data (Brohan et al., 2023), while PaLM-E incorporates visual inputs and robot states into an embodied language model (Driess et al., 2023). Socratic Models further demonstrate zero-shot reasoning by composing specialized vision and language models (Zeng et al., 2023). These methods strengthen the connection between perception and action, yet they are generally evaluated on individual robots and predefined manipulation or navigation tasks. They do not directly address resource-constrained mission planning for dynamic, interdependent tasks across a rescue fleet. Moving from an individual embodied agent to a fleet-level decision layer introduces coupling among vehicle states, task priorities, routes, and shared resources, so an action that is feasible for one vehicle may still be unsuitable for the mission as a whole.

LLM-based agents have also explored closed-loop decision making through environmental feedback, navigation, and accumulated experience. Inner Monologue uses language-based feedback to revise embodied plans (Huang et al., 2023), and LM-Nav combines large pretrained models with semantic waypoints for open-world navigation (Shah et al., 2023). Voyager couples an LLM agent with an automatic curriculum and a growing skill library for open-ended exploration (Wang et al., 2023). Nevertheless, these systems primarily target single- agent embodied environments. They establish that feedback and accumulated experience can improve sequential behavior, but do not establish how these ideas should be combined with fleet-wide resource constraints or how their importance changes as operational pressure increases in multi-UAV missions.

Taken together, the literature provides the main ingredients for high-level robot decision making, but not their integration around the specific failure modes of dynamic, resource-constrained fleet missions. An LLM-centered closed-loop decision framework for this setting must satisfy three requirements beyond generating an initial plan. It must reject plans that violate operational or tactical constraints, retrieve relevant experience when resource and scheduling interactions make planning less routine, and repair the active mission when execution feedback changes the underlying state. These requirements motivate a closed-loop framework in which Validation, Memory, and Runtime Replanning are not independent additions, but complementary mechanisms for sustaining high-level decision making from initial observation through execution.

3 Task Modeling and System Design

3.1 Task Modeling

Wildfire rescue is a closed-loop decision task rather than an isolated perception or allocation problem. In an operational mission, aerial observations first reveal dispersed fire points; these points must then be organized into actionable task zones. The fleet must allocate its limited energy, payload, and flight time to these regions, while unexpected

failures or changes in the fire situation may require the current plan to be revised during execution. We therefore abstract the real rescue workflow as a sequence from fire-situation understanding to resource-constrained mission planning and runtime replanning.

This workflow exposes three risks that organize the system design. First, a generated mission plan can appear coherent while violating tactical or structural constraints. Second, as resource and scheduling interactions become more complex, one-shot reasoning may fail to use relevant experience from previous missions. Third, a plan that is feasible initially can become obsolete after the fire, fleet, or mission state changes. High-level closed-loop decision making must therefore check candidate plans before execution, recover relevant experience when operational pressure increases, and revise the active mission after runtime perturbations. The task model below represents the information needed to support this continuing decision loop.

For each mission, the rescue world is represented by the structured state

$$S = \{O, F, R, U, C, E\}.$$

Here, O denotes aerial observations; F denotes fire points; R denotes task zones; U denotes UAV operational states; C denotes mission constraints; and E denotes runtime perturbations. This representation separates the underlying rescue world from the information available to the decision system. At mission start, the system receives $X_0 = \{O, U, C\}$, whereas F and R must be inferred from the observation. When a runtime perturbation occurs, it receives the updated operational state U_t and the observed perturbation E_t. The required outputs are an initial mission plan P and, when necessary, a revised plan P'.

The resulting task model comprises the following decision chain:

$$O \to F \to R, \qquad (R, U, C) \to P,$$
$$(P, U_t, E_t) \to P'.$$

The first two relations transform raw aerial observations into an actionable regional representation of the fire situation. The third relation generates an executable fleet plan under operational constraints. The final relation adapts that plan to changes encountered during execution. This formulation captures the continuous rescue workflow while retaining the stage-specific information needed to design and evaluate a closed-loop system.

3.2 System Overview

Based on the task model above, the proposed LLM-centered closed-loop decision framework is organized around the flow of mission state, candidate plans, corrective feedback, and prior experience. Before execution, Vision Analysis transforms aerial fire imagery into fire points and task zones. Mission Planning combines this representation with fleet states, operational constraints, and retrieved experience from Memory to produce a candidate plan. Validation then either accepts the plan or routes targeted feedback back to Mission Planning for correction. This pre-execution loop can repeat until an executable mission plan is accepted.

During execution, the accepted plan is evaluated against updated fleet states and runtime perturbations. When an event invalidates part of the plan, Runtime Replanning uses the current execution state and relevant Episodic Memory to revise the affected part. Mission traces and outcomes are subsequently stored as episodic cases or distilled into

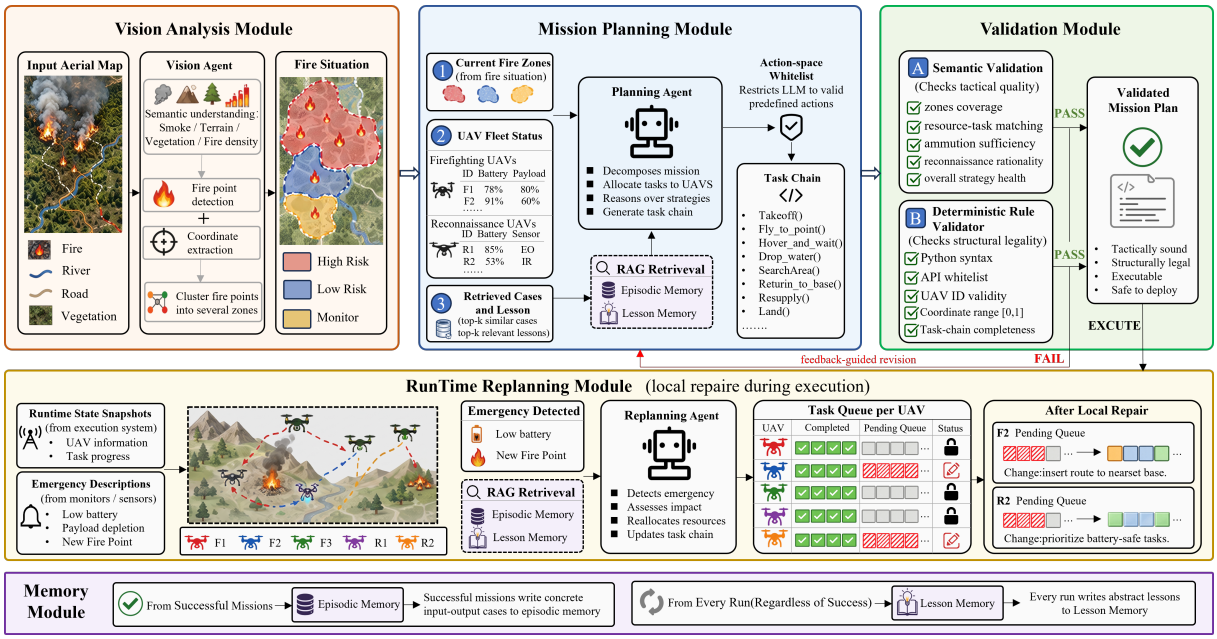


Figure 1: Overview of the proposed system.

lessons by Memory. In this way, Memory supports both initial planning and runtime replanning, while Validation and execution feedback close the decision loop at different stages. Figure 1 shows this information and feedback flow.

Vision Analysis. Vision Analysis anchors the decision loop by reducing the ambiguity between raw aerial observations and the planning state. It converts raw aerial imagery into a structured description of the fire situation. This module uses a multimodal LLM to perform two tasks in a single inference pass. First, it detects all visible fire points in the image and outputs their normalized coordinates. Second, it clusters these points by spatial density, partitioning the continuous fire field into several discrete task zones.

This module uses a multimodal LLM rather than a vision-only model because zone division depends on more than fire point locations. The division also depends on visual context, such as smoke, vegetation, and terrain boundaries. These cues require joint reasoning over appearance and spatial semantics. The resulting fire points and task zones provide the shared initial-state representation used by Mission Planning.

Mission Planning. Mission Planning addresses the fleet-level coupling among task priorities, vehicle states, and resource constraints. It is the core decision module and converts the structured fire situation produced by Vision Analysis into a candidate multi-UAV collaborative plan. A planning LLM generates a complete task chain for each UAV in the fleet, covering task allocation, temporal sequencing, and endurance management. To keep outputs controllable, the LLM is restricted to a predefined set of allowable actions.

Mission Planning combines the fire situation, retrieved experience, and operational constraints. It receives the structured fire situation from Vision Analysis, retrieves relevant past cases and lessons from both memory modules, and produces a coordinated plan in which each UAV's task chain respects the fleet's capabilities and the mission's operational constraints. Formally, the planning process is expressed as

$$P = \mathcal{G}\big(R, U, C, M_E(q), M_L(q)\big), \qquad q = q(R, U, C), \tag{1}$$

where $\mathcal{G}$ denotes the planning LLM, q is a query constructed from the current task zones, fleet states, and constraints, and $M_E(q)$ and $M_L(q)$ denote the episodic cases and lessons retrieved for that query, respectively. The resulting P is a candidate plan rather than an automatically trusted output; it is passed to Validation before mission execution.

Validation. Validation addresses the risk that a linguistically coherent candidate plan may still be tactically implausible or structurally invalid. The module assesses each candidate plan through two sequential checks. Together, the two checks form the audit–correction loop: when either one rejects a plan, its feedback is routed back to Mission Planning for revision. This loop turns validation failures into targeted feedback for the next planning attempt.

The first check, Semantic Validation, uses an independent LLM to evaluate the plan from a tactical rationality perspective. It examines zone coverage completeness, resource-task matching, payload adequacy, and overall strategic health. Using a separate LLM rather than asking the planning LLM to self-check avoids self-consistency bias.

The second check, Deterministic Rule Validation, uses a rule engine to verify structural legality, including UAV existence, coordinate validity, and task chain completeness. Unlike the semantic check, this rule check asks whether the plan is structurally legal. This question can be answered deterministically within the defined rule set. Only a plan accepted by both checks proceeds to execution, preserving the separation between plan generation and plan acceptance.

Runtime Replanning. Runtime Replanning addresses the risk that an accepted plan becomes invalid after the operational state changes. This module handles runtime perturbations during mission execution. It takes a runtime state snapshot as input, including current positions, battery levels, payloads, and task progress for each UAV. Its output is not a new global plan, but a local revision of the current plan. Unlike the main pipeline, Runtime Replanning reasons about a dynamic execution state and changes only the affected part of the plan.

When a runtime perturbation occurs, the module retrieves similar fault cases from Episodic Memory as repair references. These cases describe how comparable failures were resolved in past missions, providing the replanning LLM with tactical precedents for the current situation. The resulting local revision closes the execution-feedback loop without restarting the complete pre-execution pipeline.

Memory. Memory addresses the risk that relevant prior experience is unavailable when coupled resource, scheduling, or event conditions make the current decision less routine. It allows the system to reuse past experience. The two memory modules differ in what they store and how their contents are used.

Episodic Memory stores concrete successful cases as input–output pairs. Each case records the situation, the generated or revised plan, and the corresponding outcome. Cases are indexed by situational features, so that when a new mission situation arises, the system retrieves the most similar past cases as tactical references. Specifically, the episodic memory is represented as

$$M_E = \{(z_i, P_i, o_i)\}_{i=1}^{N_E}, \tag{2}$$

where z_i, P_i, and o_i denote the task situation, the associated plan, and the execution outcome of case i, respectively. For either episodic memory or lesson memory, the system

retrieves the K most semantically similar entries to the current query according to

$$\mathcal{R}_K(q, M) = \underset{m_i \in M}{\text{TopK}}\ \text{sim}(q, m_i), \qquad \text{sim}(q, m_i) = \frac{\phi(q)^\top \phi(m_i)}{\|\phi(q)\|_2 \|\phi(m_i)\|_2}, \tag{3}$$

where $M \in \{M_E, M_L\}$ is the queried memory, $\phi(\cdot)$ maps a query or memory entry to its semantic embedding, and $\text{sim}(\cdot, \cdot)$ is the cosine similarity. The retrieved set is supplied to the corresponding planning or replanning prompt as reference information.

Lesson Memory serves a different role. Instead of storing concrete cases, it keeps higher-level lessons distilled from every run. After each mission, an LLM reviews the full execution trace. The trace includes the initial plan, validation feedback, runtime perturbations, replanning actions, and final scores. The LLM then extracts concise, reusable lessons. These lessons capture patterns such as common planning mistakes, resource estimation biases, and effective repair strategies. Together, the two forms of Memory connect prior outcomes to current decisions: Episodic Memory supplies concrete tactical precedents, whereas Lesson Memory supplies reusable guidance about recurring planning and resource-management patterns.

4 Evaluation Suite Design

The evaluation suite is designed to test whether high-level closed-loop decision making with the LLM-centered closed-loop decision framework remains effective as operational pressure increases. This requires more than evaluating perception, planning, or replanning separately. Existing datasets and evaluation settings only partially cover the closed-loop rescue decision chain defined in Section 3. As summarized in Table 1, no existing setting jointly evaluates the transition from aerial observation to task-zone construction, resource-constrained mission planning, and runtime replanning. This comparison concerns evaluation coverage rather than algorithmic performance.

Table 1: Coverage of evaluation capabilities in existing datasets, benchmarks, and the evaluation suite used in this work. •indicates explicit coverage, ◦indicates partial coverage, and —indicates no explicit coverage.

Benchmark / Dataset	**Forest Fire Perception**	**Disaster Semantics**	**UAV-level Constraints**	**Runtime Replanning**
DOTA (Xia et al., 2018)	•	—	—	—
VisDrone (Zhu et al., 2022)	•	—	—	—
UAVs-FFDB (Mowla et al., 2024)	•	•	—	—
FLAME (Shamsoshoara et al., 2021)	•	•	—	—
MS-FSDB (Han et al., 2024)	◦	•	—	—
WildFireVQA (Habibpour et al., 2026)	•	•	◦	—
RMASBench (Kleiner et al., 2013)	—	•	◦	◦
CrisisBench (Alam et al., 2021)	—	•	—	—
Our Evaluation Suite	•	•	•	•

To close this gap, we instantiate the task model of Section 3 as a controlled, real-UAV-informed evaluation suite in AirSim. The suite evaluates whether a system can complete the full rescue loop within a single episode, from fire-situation understanding to an executable plan and runtime replanning after perturbations. The suite separately exposes task completion, execution quality, and mission continuity, preventing high task

completion from being interpreted as uniformly effective execution under pressure. Its design combines three complementary elements.

First, real-to-sim calibration constrains simulated platforms, sensing, and mission conditions by the capabilities and safety limits of a real UAV. The resulting evaluation episodes are therefore physically and operationally grounded rather than arbitrarily configured. Second, difficulty stratification varies the complexity of fire-point layouts, fleet-resource conditions, scheduling interactions, and runtime perturbations while preserving a common task definition. It enables controlled comparison of models and system mechanisms as operational pressure increases. Third, mission-level metrics assess the outcome of the complete rescue process from complementary operational perspectives, rather than collapsing performance into a single score. The following sections detail these three elements.

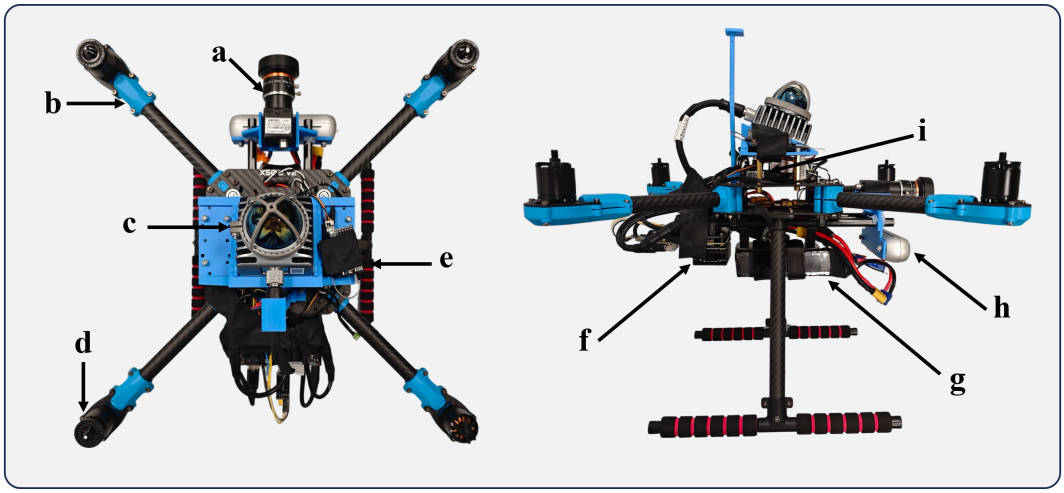


Figure 2: UAV platform and its main components:(a) LiDAR, (b) frame, (c) monocular camera, (d) motor, (e) receiver, (f) onboard computer, (g) power supply, (h) stereo camera, and (i) flight controller.

4.1 Real-to-Sim Calibration

The first design property is realized through a co-design between the real UAV platform and the AirSim simulation environment. As stated above, real UAV parameters constrain every evaluation episode. Accordingly, the real platform defines the engineering boundary of the rescue mission, while AirSim configures controllable and repeatable evaluation episodes within that boundary. Rather than treating the two as separate experimental settings, we use the real platform to define the feasible set from which AirSim configures each simulated UAV and its environment. For each UAV, the configured mission must satisfy

$$\mathcal{E}_{\mathrm{UAV}} = \left\{ (h, v, m, e_{\mathrm{avail}}, d) \;\middle|\; \begin{array}{ll} h \in \mathcal{H}_{\mathrm{safe}}, & v \in \mathcal{V}_{\mathrm{safe}}, \\ m \le m_{\max}, & e_{\mathrm{req}} + e_{\mathrm{res}} \le e_{\mathrm{avail}}, \\ d \le d_{\mathrm{link}} & \end{array} \right\}, \tag{4}$$

where h, v, m, e_{avail}, and d denote flight altitude, velocity, payload mass, available energy, and distance to the coordination link. The safe altitude and velocity ranges are specified by the operating policy. The payload and energy bounds are grounded in the airframe, propulsion system, and battery; the energy reserve enforces a safe return; and

the link bound is determined by the deployed communication configuration. Thus, the numerical bounds combine hardware specifications, flight-safety policy, and deployment configuration. Any configuration that violates Eq. 4 is excluded before episode generation. The real UAV platform is shown in Figure 2, and its hardware specifications are listed in Table 2.

Table 2: Technical specifications of the real UAV platform.

Subsystem	**Model**	**Key specifications**
UAV frame	Holybro X500 V2	Carbon-fiber quadrotor frame, 500-mm wheelbase, approximately 610-g frame weight, and 1.5-kg published payload capacity excluding the battery at 70% throttle.
Flight controller	Holybro Pixhawk 6X	STM32H753 processor at 480 MHz, triple-redundant IMUs, dual barometers, 16 PWM outputs, two CAN buses, and 100-Mbps Ethernet.
Companion computer	NVIDIA Jetson Orin NX 8 GB	6-core Arm Cortex-A78AE CPU, NVIDIA Ampere GPU, 8-GB LPDDR5 memory, up to 117 TOPS, and configurable 10–40-W power modes.
LiDAR	Livox Mid-360	360° horizontal and −7°–52° vertical field of view, 200,000 points/s, 10-Hz frame rate, 40-m range at 10% reflectivity, 0.1-m blind zone, 265-g weight, and 6.5-W average power.
RGB-D camera	Intel RealSense D455	Global-shutter stereo depth camera, 87° × 58° depth field of view, depth resolution up to 1280 × 720 at 90 fps, approximately 0.6–6-m effective range, integrated IMU, USB 3.1 interface, and 116-g weight.
Propulsion system	4× Holybro 2216 KV920	Four 920-KV brushless motors, four 20-A ESCs, and 10 × 4.5-in propellers, with a maximum manufacturer bench thrust of approximately 1.33 kgf per motor at 16 V.
Battery	Dualsky XP3300-6HED	6S lithium-polymer battery, 3300-mAh capacity, 22.2-V nominal voltage, 25.2-V fully charged voltage, and approximately 73.3-Wh nominal energy.

The virtual observation process is further fixed by the real camera geometry. Under a nadir-view, locally planar-ground approximation, for altitude h, horizontal field of view θ_{FOV}, and horizontal resolution N_x, the image footprint and ground sampling distance are

$$W(h) = 2h \tan\left(\frac{\theta_{\mathrm{FOV}}}{2}\right), \qquad \mathrm{GSD}(h) = \frac{W(h)}{N_x}. \tag{5}$$

Thus, the AirSim camera pose and the scale, density, and placement of fire points are jointly constrained by what the real camera could observe, rather than selected independently. Energy and payload limits likewise constrain route length, task duration, and suppressible workload. Table 3 records these calibration rules and the resulting simulation settings.

These calibrated bounds provide the common operational basis on which the difficulty levels are constructed.

Table 3: Real-to-sim calibration of AirSim UAV and environment settings.

Calibration target	**Real-platform basis**	**AirSim configuration**
Imaging geometry	Operating altitude and the RGB-D camera field of view and resolution	Observation height h, camera pose, field of view, and resolution; the resulting footprint $W(h)$ and GSD determine the observable ground area and image scale.
Energy model	Battery capacity, propulsion characteristics, and return-to-base reserve policy	Available energy e_{avail}, reserve e_{res}, and energy consumption for flight and task execution.
Payload model	Airframe and propulsion payload limit	Maximum payload $m_{\max}$ and the onboard suppressant inventory for each simulated UAV.
Communication model	Deployed communication configuration and operating range	Link-distance threshold d_{link} and the distance-dependent connectivity condition.

4.2 Difficulty Stratification

The second design property is that each evaluation episode is purpose-built rather than randomly sampled. Each episode instantiates the task variables defined in Section 3 through three controlled dimensions: fleet and resource conditions, fire-point layouts, and, where applicable, runtime perturbations. Their combinations define four stress-testing difficulty levels in Table 4. Each difficulty level is instantiated by a fixed set of 20 evaluation episodes, yielding 80 episodes in total.

The four levels form a staged operational-pressure axis rather than four unrelated scenario groups. The simple and normal levels test fire-situation understanding and resource-constrained mission planning under progressively tighter initial conditions. The hard and expert levels retain these pressures while adding runtime perturbations and stronger scheduling interactions. This progression makes episode difficulty useful not only for measuring end-to-end degradation, but also for identifying when Validation, Memory, and Runtime Replanning become consequential.

Table 4: Four difficulty levels and their controlled episode conditions.

Difficulty levels	**Fire-point configuration**	**Fleet/resource conditions**	**Runtime perturbations**
Simple	Sparse static fire points; few task zones.	Ample energy and payload.	None.
Normal	Multiple fire points and task zones.	Multi-UAV allocation; normal resource margin.	None.
Hard	Multi-region fire scene.	Energy or payload constrained.	Few, temporally separated perturbations.
Expert	Dense or spreading fire-point layout.	Competing resource demands.	Frequent perturbations; global scheduling pressure.

The fleet and resource conditions in every evaluation episode are constrained by the real-to-sim calibration in Section 4.1. In particular, each simulated UAV must satisfy the feasible operating envelope in Eq. 4, and its imaging, energy, payload, and communication settings follow Table 3. Difficulty is therefore varied only within the engineering and operational limits established for the real platform.

The fire-point configurations of all episodes are pre-specified rather than randomly generated. These configurations control the scene structure and the spatial arrangement and number of fire points per task zone across difficulty levels. Figure 3 provides representative scenes from the four difficulty levels, and Figure 4 summarizes the distribution of fire-point counts per task zone within each difficulty level. Together, they illustrate the scene layouts and task-zone-level fire-point distributions used to instantiate the four difficulty levels.

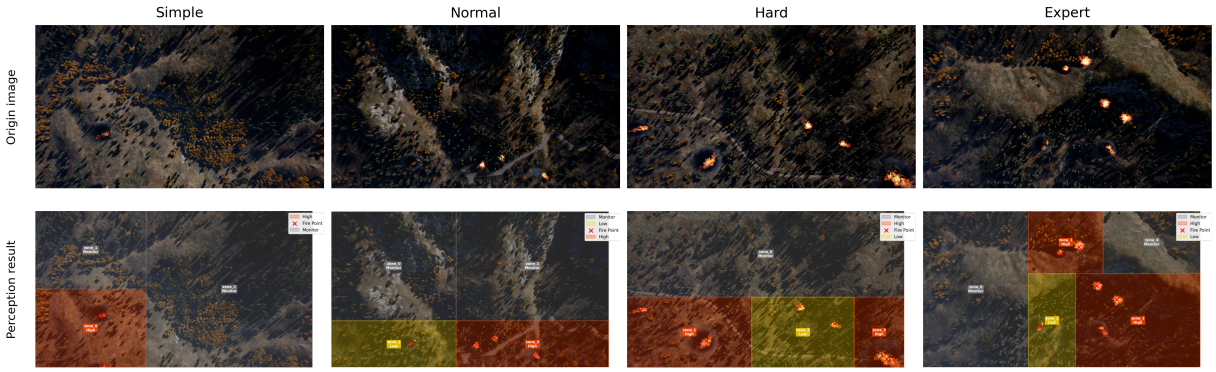


Figure 3: Representative fire scenes from the four difficulty levels. In each example, *Origin image* denotes the original aerial image, and *Perception result* denotes the output generated by the Vision Analysis module, which uses Gemini 3.1 Pro as its foundation model.

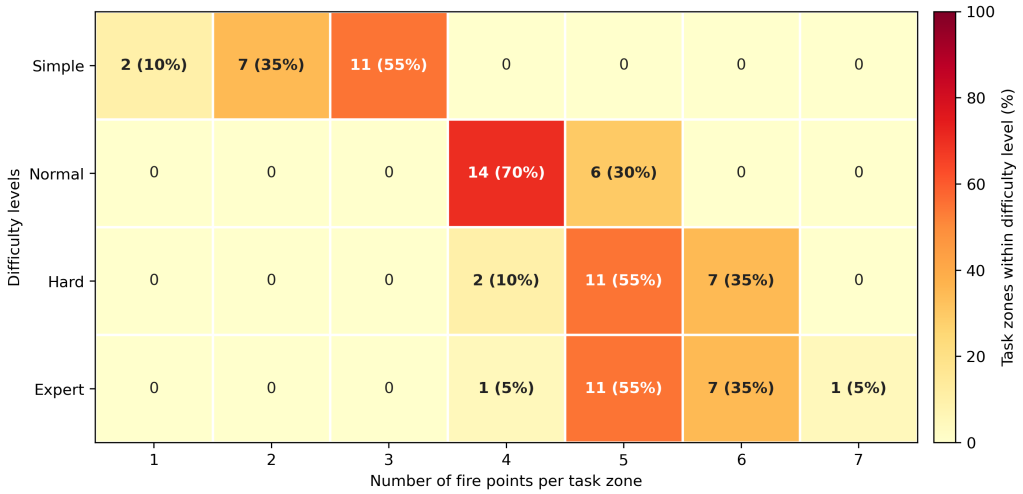
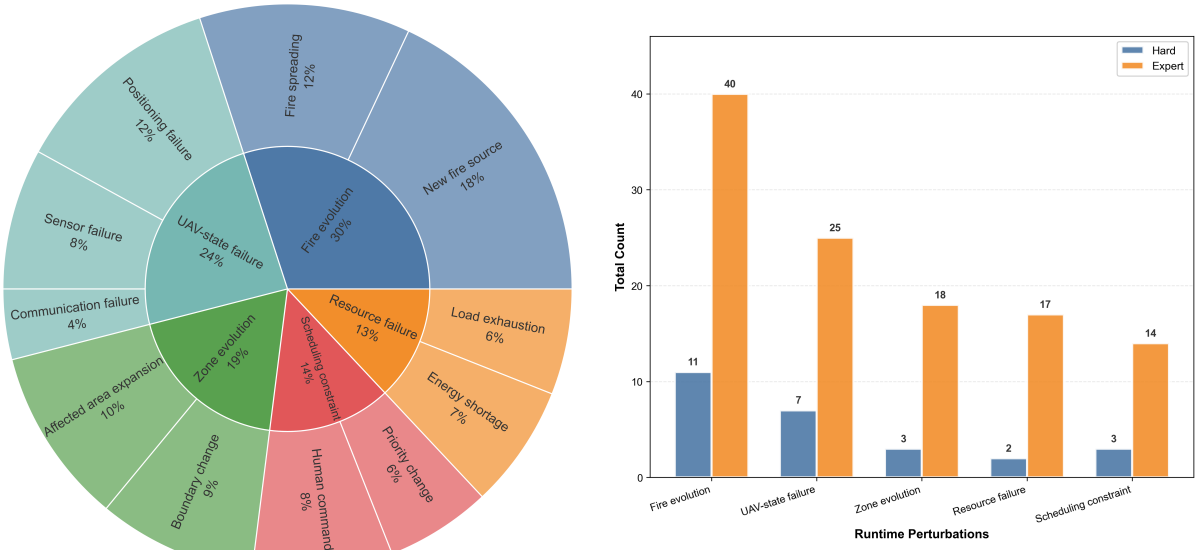


Figure 4: Distribution of fire-point counts per task zone across difficulty levels.

The third controlled dimension introduces runtime perturbations after mission execution has begun. Unlike the preceding two dimensions, which specify the initial fleet state and fire scene, these events modify the operational state or mission constraints faced by an active plan. For this evaluation suite, we construct our own runtime perturbation library comprising five families: fire evolution (30%), UAV-state failure (24%), zone evolution (19%), scheduling constraint (14%), and resource failure (13%). As shown in Figure 5(a), each family contains concrete event types, such as new fire sources and fire spreading, positioning, sensor, and communication failures, boundary changes and affected-area expansion, human commands and priority changes, and load exhaustion and energy shortage.

Runtime perturbations are enabled only in the hard and expert levels and are placed at predefined points in each episode so that replanning behavior can be evaluated under repeatable conditions. During suite construction, events are drawn once from the library according to the configured sampling distribution and then fixed within the evaluation episodes. Hard episodes contain a small number of temporally separated perturbations, whereas expert episodes contain more frequent perturbations that may occur concurrently or in succession and impose stronger communication and global scheduling pressure. Figure 5(b) shows the resulting draws by perturbation family: expert episodes contain more events in every family, while preserving coverage of all five families.



(a) Runtime perturbation library.

(b) Distribution of perturbations in hard and expert levels.

Figure 5: Runtime perturbations used for closed-loop evaluation.

4.3 Evaluation Metrics

We evaluate end-to-end behavior using six complementary mission-level metrics: completion, response, mission time, event stability, resource pressure, and coordination. They measure objective attainment, response timeliness, execution efficiency, continuity under perturbations, resource use, and workload balance, respectively. Event stability is evaluated only in the hard and expert settings, where runtime perturbations are introduced.

All scores are normalized to $[0, 100]$, with higher values indicating better performance. Let Z be the set of task zones, $c_z \in [0, 1]$ the completion ratio of zone z, and ω_z its priority weight. Let $\bar{t}_{\text{resp}}$ and τ denote the priority-weighted mean time to first effective intervention and the total mission duration. The six scores are defined jointly as

$$
\begin{aligned}
s_{\text{comp}} &= 100 \frac{\sum_{z \in Z} \omega_z c_z}{\sum_{z \in Z} \omega_z}, & s_{\text{resp}} &= 100 \left[1 - \left[\frac{\bar{t}_{\text{resp}} - l_{\text{resp}}}{u_{\text{resp}} - l_{\text{resp}}} \right]_0^1 \right], \\
s_{\text{time}} &= 100 \left[1 - \left[\frac{\tau - l_{\text{time}}}{u_{\text{time}} - l_{\text{time}}} \right]_0^1 \right], & s_{\text{stab}} &= \frac{100}{N_e} \sum_{e=1}^{N_e} \mathbb{I}\big(\text{Recovered}(e)\big), \\
s_{\text{res}} &= 100 \left[1 - \frac{\bar{p} + \max_j p_j}{2} \right], & s_{\text{coord}} &= 100 \left[1 - \frac{G(\mathbf{n}) + G(\mathbf{d}) + G(\boldsymbol{\tau})}{3} \right].
\end{aligned}
\tag{6}
$$

Here, $[x]_0^1 = \min\{1, \max\{0, x\}\}$ clips a value to $[0, 1]$; l_{resp}, u_{resp}, l_{time}, and u_{time} are fixed time-reference thresholds. N_e is the number of runtime perturbations, and Recovered(e) indicates recovery by a feasible revision within the allowed window. $p_j \in [0, 1]$ and $\bar{p}$ denote the per-UAV and fleet-average resource pressure. $G(\cdot) \in [0, 1]$ is the Gini coefficient of the assigned task counts $\mathbf{n}$, flight distances $\mathbf{d}$, or execution durations $\boldsymbol{\tau}$.

Because no perturbations are introduced in the simple and normal settings, their applicable metric set $\mathcal{K}_d$ contains completion, response, mission time, resource pressure, and coordination; for the hard and expert settings, it additionally contains event stability. For each episode i at difficulty level d, the score combines the weighted aggregation of its applicable metrics with a bounded difficulty reward. The reported score for level d is the mean across its N_d episodes:

$$S_d = \frac{1}{N_d} \sum_{i=1}^{N_d} \left[\sum_{k \in \mathcal{K}_d} w_{k,d} s_{i,k} \frac{1 + \lambda \rho_i}{1 + \lambda} \right], \qquad w_{k,d} \geq 0, \quad \sum_{k \in \mathcal{K}_d} w_{k,d} = 1, \tag{7}$$

where $w_{k,d}$ is the normalized weight of metric k at difficulty level d, $\rho_i \in [0, 1]$ is the difficulty coefficient of episode i, and $\lambda \geq 0$ controls the strength of the difficulty reward. The normalization by $1 + \lambda$ ensures that $S_d \in [0, 100]$. The component scores remain available for diagnostic analysis, while S_d provides a unified measure for overall comparison.

Together, the staged episode difficulty and decomposed metrics provide a common basis for testing the three questions examined next: whether the complete framework sustains high-level closed-loop decision making, which mechanisms matter as operational pressure increases, and how much capability for resource-constrained mission planning can be retained by a compact planner.

5 Experiments

5.1 Experimental Setup

We evaluate the proposed system using the evaluation suite described in Section 4. The three experimental settings examine three questions in sequence. The first evaluates whether the complete framework can execute the end-to-end decision loop across different downstream foundation-model configurations as operational pressure increases. The second examines how Validation, Memory, and Runtime Replanning address their respective decision risks under increasing operational pressure. The third assesses whether validated decision traces can specialize a lower-overhead compact planner for resource-constrained mission planning.

To isolate the influence of the foundation model on Mission Planning, Validation, and Runtime Replanning, we fix Gemini 3.1 Pro as the foundation model for Vision Analysis in the first setting. We then evaluate four configurations by using ChatGPT 5.5, Gemini 3.1 Pro, Claude Opus 4.7, and DeepSeek V4 Pro, respectively, in the remaining LLM-based modules. This design keeps the transformation from aerial imagery to the structured fire situation consistent across configurations, allowing their subsequent differences to reflect the models used in Mission Planning, Semantic Validation, and Runtime Replanning. The four configurations are controlled variants of the proposed system rather than external algorithmic baselines; because Vision Analysis is fixed, the comparison concerns the downstream decision modules. Each configuration is evaluated on the same 80

episodes, comprising 20 episodes at each of the simple, normal, hard, and expert difficulty levels.

For the second setting, we use the DeepSeek V4 Pro configuration and compare the full system with three ablated variants: w/o Validation, w/o Memory, and w/o Runtime Replanning. The first removes the semantic and deterministic audit–correction loop; the second disables retrieval from both Episodic Memory and Lesson Memory; and the third retains the initial plan after runtime perturbations without invoking Runtime Replanning. All other models, prompts, and execution settings remain unchanged. All four configurations are evaluated on the same 80 episodes, with 20 episodes at each difficulty level. In the simple and normal settings, where no runtime perturbations occur, the w/o Runtime Replanning variant differs only in that the Runtime Replanning module is disabled and therefore remains inactive throughout execution.

For the third setting, we construct 2,000 mission-planning demonstrations using the proposed system with Gemini 3.1 Pro as the teacher planner. The demonstrations are generated from mission situations disjoint from the 80 evaluation episodes. Each demonstration has the form

$$(R, U, C) \rightarrow P,$$

where each demonstration pairs a structured description of the task zones, UAV states, and mission constraints with the final multi-UAV plan accepted after the Semantic Validation and Deterministic Rule Validation loop. We use these validated demonstrations to specialize Qwen3-8B for the planning mapping defined in Section 3. The dataset is divided into 90% for training and 10% for validation. We apply 4-bit quantized low-rank adaptation with a maximum sequence length of 6,144 tokens. The model is trained for three epochs with an effective batch size of 16 and a learning rate of 5×10^{-5}. Fine-tuning is conducted on a single NVIDIA GeForce RTX 4090, with 16 Intel Xeon Gold 6430 vCPUs and 120 GB of system memory.

We evaluate both the original and fine-tuned Qwen3-8B by replacing only the planner used by Mission Planning. Vision Analysis and Semantic Validation continue to use Gemini 3.1 Pro, while Deterministic Rule Validation and all other non-LLM components remain unchanged. The two Qwen3-8B configurations are evaluated on the same 40 simple and normal episodes used in the first setting, with 20 episodes at each level. Because these levels contain no runtime perturbations, this experiment does not invoke Runtime Replanning or assess event stability. The comparison therefore isolates the effect of planner fine-tuning on the same episode subset.

5.2 End-to-End Evaluation

Before examining individual mechanisms, we first evaluate the complete system across multiple foundation-model configurations. This experiment establishes end-to-end task performance under the proposed evaluation setting and examines the sensitivity of the downstream decision process to model choice. Its primary purpose is to determine whether the full decision loop can be executed across the evaluated configurations, rather than to produce a general ranking of foundation models. Table 5 reports the results across all four difficulty levels, while Figure 6 summarizes the overall scores. All configurations achieve complete task coverage in the simple and normal episodes, near-complete coverage in the hard episodes, and completion scores above 90 in the expert episodes. Thus, each evaluated configuration can support the full decision loop from the shared Vision Analysis

output to mission execution, although the quality and continuity of execution vary with operational pressure.

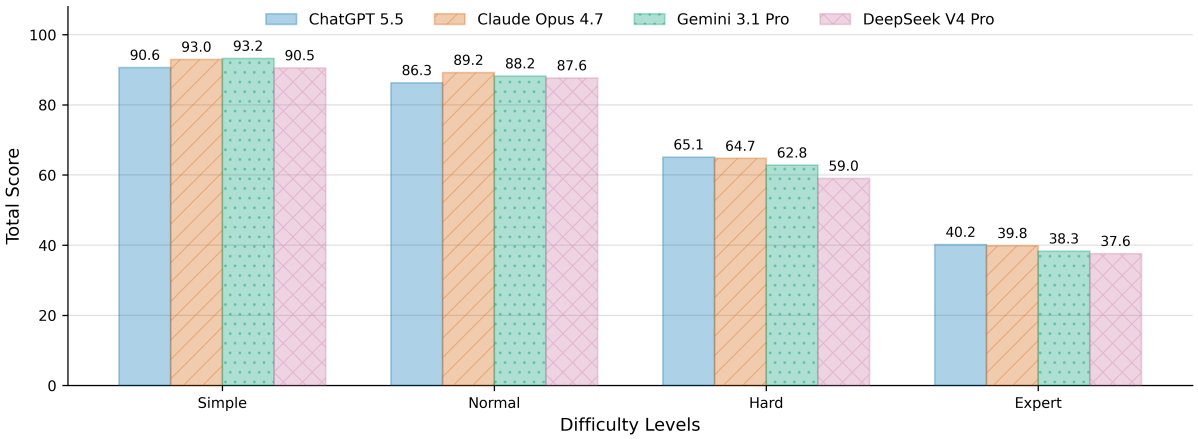


Figure 6: Overall scores of the four foundation-model configurations across episode difficulty levels.

Table 5: End-to-end results across foundation-model configurations and episode difficulty levels. S is the overall score, and the remaining columns report normalized mission-level scores in $[0, 100]$ (higher is better). Event stability is not applicable to the simple and normal settings and is denoted by "–". The highest observed value for each metric at every difficulty level is shown in bold when it is unique; these highlights describe the evaluated configurations rather than a general ranking of foundation models.

Difficulty	Foundation model	S	Mission-level score					
			Completion	Response	Time	Event stability	Resource pressure	Coordination
Simple	ChatGPT 5.5	90.63	100.00	89.18	78.99	–	81.20	79.54
	Gemini 3.1 Pro	**93.23**	100.00	88.78	**91.06**	–	**84.45**	80.71
	Claude Opus 4.7	92.97	100.00	**91.44**	86.62	–	82.18	**83.96**
	DeepSeek V4 Pro	90.54	100.00	87.66	78.90	–	83.05	80.04
Normal	ChatGPT 5.5	86.28	100.00	93.82	66.27	–	77.64	73.19
	Gemini 3.1 Pro	88.23	100.00	94.10	71.50	–	**86.68**	71.72
	Claude Opus 4.7	**89.18**	100.00	**96.09**	68.24	–	84.54	**78.12**
	DeepSeek V4 Pro	87.63	100.00	91.36	**71.74**	–	83.95	75.13
Hard	ChatGPT 5.5	**65.09**	100.00	**94.97**	**41.22**	**42.25**	**63.19**	**67.24**
	Gemini 3.1 Pro	62.81	100.00	92.58	39.60	33.75	59.10	60.65
	Claude Opus 4.7	64.74	99.86	94.22	35.28	40.50	62.15	65.88
	DeepSeek V4 Pro	59.03	98.57	83.90	30.76	34.75	61.36	59.07
Expert	ChatGPT 5.5	**40.15**	**93.24**	**67.01**	**13.23**	**2.67**	**60.15**	**61.25**
	Gemini 3.1 Pro	38.28	90.09	61.30	12.02	0.00	58.72	55.82
	Claude Opus 4.7	39.81	92.43	65.55	9.13	0.00	58.99	60.11
	DeepSeek V4 Pro	37.55	90.95	61.09	3.54	0.00	53.43	54.43

The overall scores are closely grouped in the simple setting (90.54–93.23) and the normal setting (86.28–89.18). The spread increases in the hard setting (59.03–65.09) and narrows again once every configuration encounters the severe expert-level conditions (37.55–40.15). The highest observed score is obtained by Gemini 3.1 Pro in simple episodes, Claude Opus 4.7 in normal episodes, and ChatGPT 5.5 in hard and expert episodes. Because this ordering changes with difficulty, the results do not yield a single model

ranking. Instead, the differences arise primarily in execution quality: response, mission time, resource pressure, coordination, and recovery from runtime events become more discriminative as operational pressure increases.

Figure 7 further compares the six mission-level metrics. In the simple and normal settings, the model profiles largely overlap near the outer region of the plots, although differences in time efficiency and coordination are already visible. As operational pressure increases, the profiles contract and differences become clearer in mission time, coordination, resource pressure, and event stability. The resulting variation shows that model choice affects how the downstream modules manage execution under higher operational pressure, even though task completion remains similar across configurations.

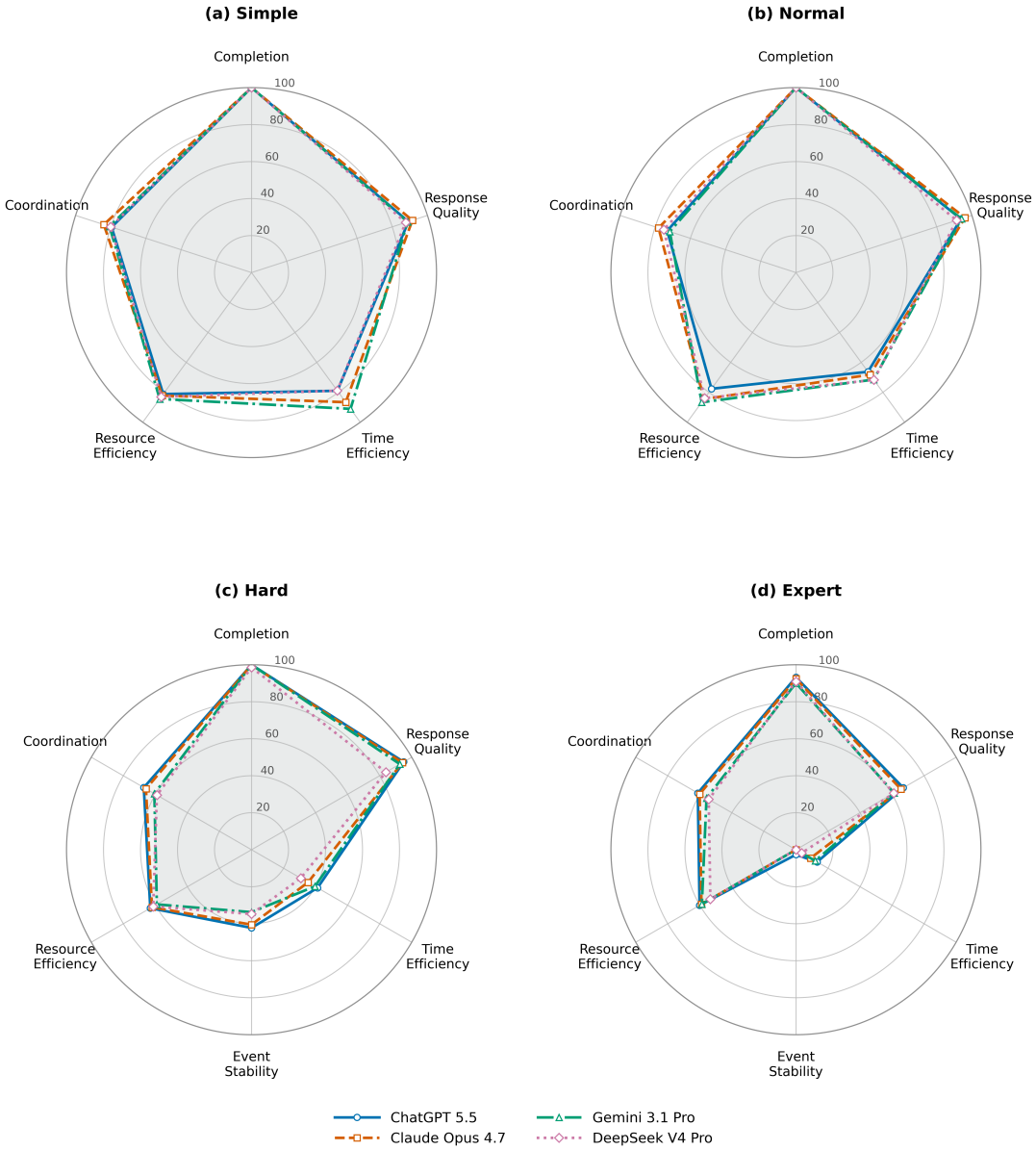


Figure 7: Mission-level performance profiles of the four foundation-model configurations at each difficulty level. Larger radial values indicate better performance.

Increasing operational pressure across the difficulty levels nevertheless degrades every configuration. The most pronounced changes occur in mission time and event stability: time scores fall to 3.54–13.23 in expert episodes, while event-stability scores fall to 0–2.67.

By contrast, expert-level completion remains above 90 for all models. Thus, the principal challenge in the hardest episodes is not whether the system can eventually complete the rescue tasks, but whether it can do so efficiently while preserving mission continuity under repeated runtime perturbations. Changing the foundation model alone does not prevent this common degradation, exposing clear headroom for more robust replanning and time-aware resource allocation. Taken together, the results show that the complete decision loop maintains high basic task completion across the evaluated downstream models, rather than depending on a single downstream foundation model. At the same time, the common decline in execution quality and mission continuity establishes the operational-pressure regime in which the roles of the closed-loop mechanisms must be examined.

5.3 Ablation Studies

Having established the end-to-end behavior of the complete framework, we next examine how its three closed-loop mechanisms address different decision risks. The ablations test whether Validation protects candidate-plan reliability, whether Memory becomes more useful as resource and scheduling interactions intensify, and whether Runtime Replanning preserves mission continuity after the execution state changes.

Table 6 reports the ablation results with DeepSeek V4 Pro fixed as the foundation model. Removing Validation causes a modest but consistent reduction across all four difficulty levels, with drops of 2.16, 2.33, 3.81, and 2.05 points from simple to expert. The limited change in average mission score does not imply that the module remains inactive: the validation loop is triggered 5, 5, 4, and 6 times in the simple, normal, hard, and expert settings, respectively. These interventions suggest that Validation primarily serves as an occasional reliability safeguard against invalid or tactically implausible plans rather than as a continuous source of score improvement.

Table 6: Ablation results with DeepSeek V4 Pro as the foundation model. Values report the mean overall score S over 20 episodes at each difficulty level. The best result in each column is shown in bold.

Configuration	**Overall score S**			
	Simple	**Normal**	**Hard**	**Expert**
Full	**90.54**	87.63	**59.03**	**37.55**
w/o Validation	88.38	85.30	55.22	35.50
w/o Memory	88.57	86.94	49.35	22.10
w/o Runtime Replanning	90.33	**88.14**	35.86	12.98

Figure 8 visualizes the score change of each ablated system relative to the full configuration.

The contribution of Memory becomes more pronounced as operational pressure increases. Removing both memory modules reduces the score by only 1.97 points in the simple setting and 0.69 points in the normal setting, but the reduction grows to 9.68 points in hard episodes and 15.45 points in expert episodes. This pattern indicates that retrieved cases and distilled lessons are most useful when resource constraints, event interactions, and scheduling pressure make planning less routine.

Disabling Runtime Replanning has little effect in the simple and normal settings, where no perturbations are triggered. In contrast, its score falls from 59.03 to 35.86

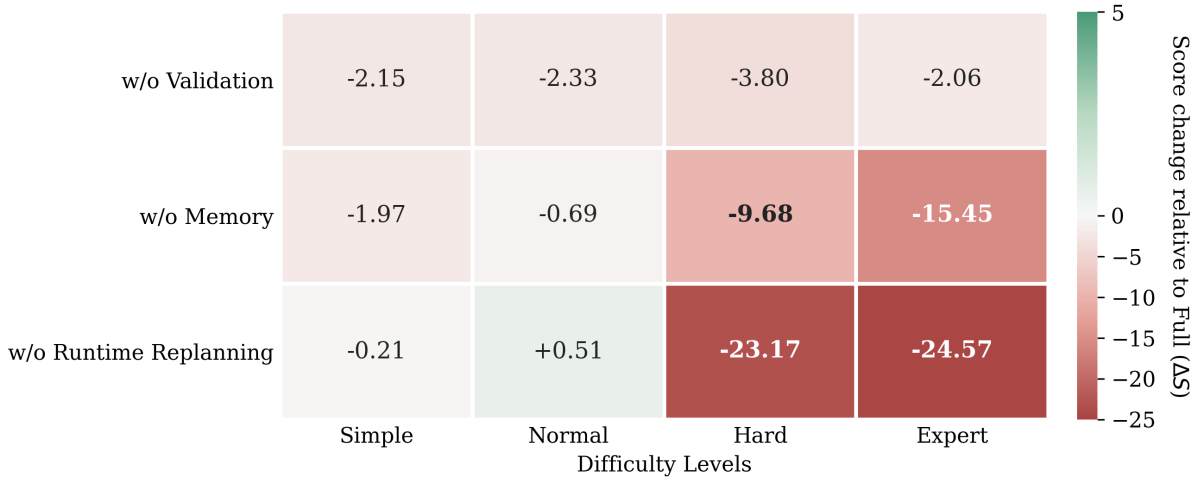


Figure 8: Change in overall score relative to the full system for each ablated variant and difficulty level. Negative values indicate performance degradation, while positive values indicate improvement.

in hard episodes and from 37.55 to 12.98 in expert episodes. The sharp degradation confirms that an initially valid plan is insufficient once the execution state changes, and that Runtime Replanning is the key mechanism for maintaining mission performance under perturbations.

The ablations therefore do not indicate three mechanisms that contribute uniformly in every episode. Validation acts as a low-frequency safeguard against invalid candidate plans, Memory contributes most when operational pressure makes planning less routine, and Runtime Replanning becomes critical when runtime perturbations invalidate an accepted plan. Their condition-dependent effects explain why all three are retained in the closed-loop framework despite their different average score contributions.

5.4 Fine-Tuned Planner Evaluation

The final experiment concerns deployment of Mission Planning.It asks whether validated demonstrations can specialize a compact planner for the simple and normal conditions in which Runtime Replanning is not invoked.

Table 7 compares the Gemini 3.1 Pro teacher planner with the original and fine-tuned Qwen3-8B planners. Fine-tuning substantially improves the compact planner on both difficulty levels. The overall score increases from 63.77 to 90.87 in the simple setting and from 67.19 to 87.39 in the normal setting. Averaged over the two levels, fine-tuning yields an absolute gain of 23.65 points and raises Qwen3-8B to 98.24% of the Gemini 3.1 Pro teacher planner's average score.

Figure 9 shows that the improvement extends across the mission-level performance profile. The original Qwen3-8B exhibits its largest deficits in mission time, resource pressure, and coordination, whereas fine-tuning substantially expands its profile toward that of Gemini 3.1 Pro. The fine-tuned planner restores near-complete task coverage and slightly exceeds Gemini 3.1 Pro in simple-level response and normal-level mission time and coordination. This indicates that the validated demonstrations transfer not only output-format compliance but also task-allocation and resource-management behavior to the compact planner.

Table 7: Comparison of the Gemini 3.1 Pro teacher planner, the original Qwen3-8B planner, and the fine-tuned Qwen3-8B planner. Mission-level scores are normalized to $[0, 100]$ and higher is better. For planning time and token usage, lower is better. The best value at each difficulty level is shown in bold when it is unique.

Difficulty	Planner	S	Mission-level score					Planning overhead	
			Completion	Response	Time	Resource pressure	Coordination	Time (s)	Tokens
Simple	Gemini 3.1 Pro	**93.23**	100.00	88.78	**91.06**	**84.45**	**80.71**	33.47	11,730
	Qwen3-8B	63.77	93.34	85.48	36.77	38.33	51.07	18.37	3,804
	Qwen3-8B-SFT	90.87	100.00	**91.44**	79.83	79.66	74.73	**12.95**	**3,485**
Normal	Gemini 3.1 Pro	**88.23**	**100.00**	**94.10**	71.50	**86.68**	71.72	41.53	14,975
	Qwen3-8B	67.19	82.50	75.17	41.50	56.91	41.78	17.52	3,844
	Qwen3-8B-SFT	87.39	98.75	90.57	**75.83**	85.45	**73.43**	**15.07**	**3,701**

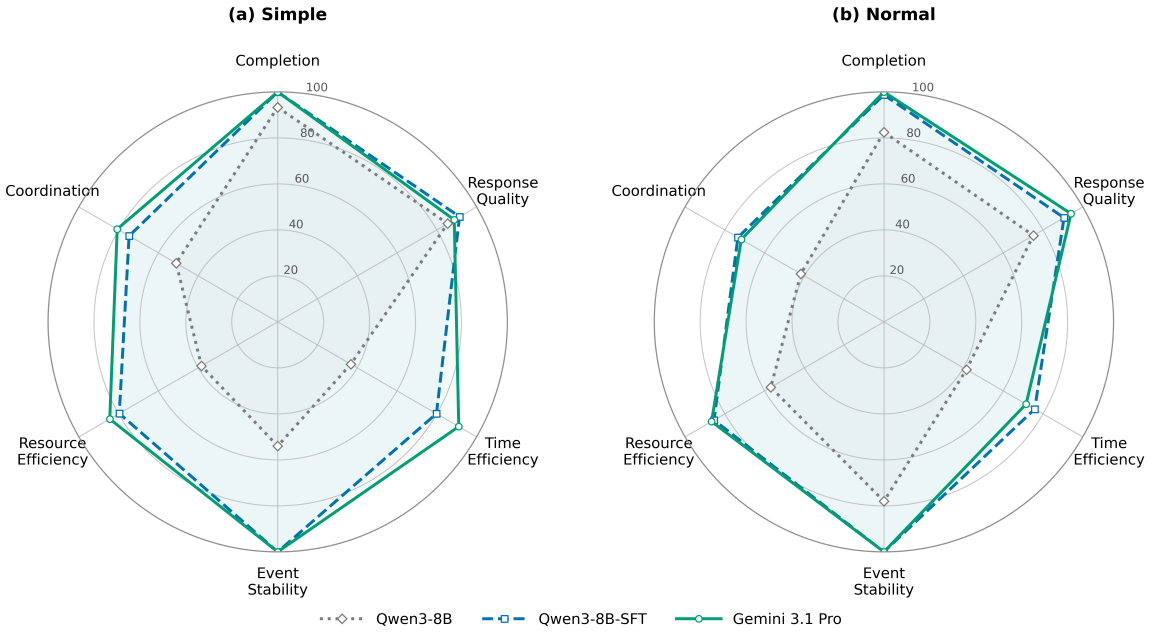


Figure 9: Mission-level profiles of the three planners in the simple and normal settings. Larger radial values indicate better performance. Event stability is not interpreted because neither setting contains runtime perturbations.

The fine-tuned planner also reduces planning overhead under the evaluated deployment setting. Its mean planning time across the two levels is 14.01 s, compared with 37.50 s for Gemini 3.1 Pro, while its mean planning-token usage falls from 13,353 to 3,593. At the individual difficulty levels, this corresponds to planning-time reductions of 61.3% and 63.7%, and token reductions of 70.3% and 75.3%, for simple and normal episodes, respectively. Figure 10 summarizes this performance–efficiency trade-off: fine-tuning closes most of the score gap to the teacher while reducing both planning latency and token usage. These results show that domain specialization enables Qwen3-8B to retain most of the teacher planner's mission performance while providing a more efficient locally deployable alternative for resource-constrained mission planning. This evidence is limited to initial planning on the evaluated simple and normal episodes; it does not establish how the compact planner would perform with Runtime Replanning under hard or expert conditions.

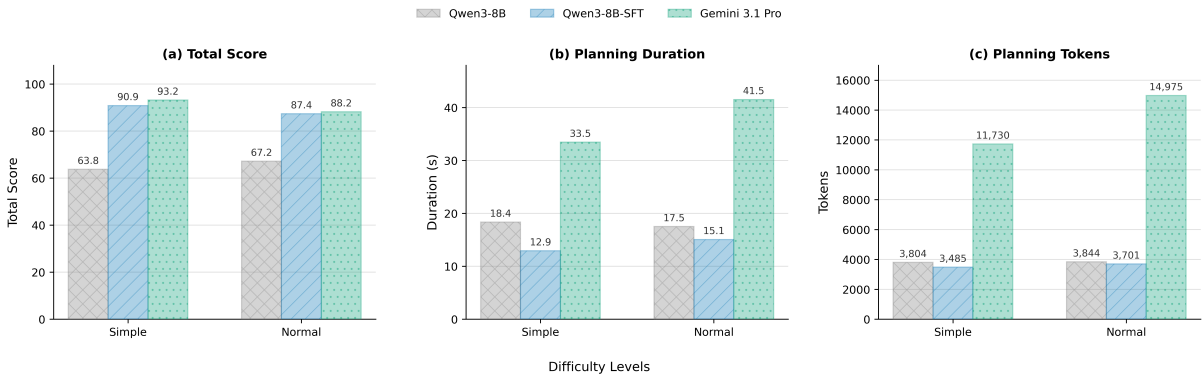


Figure 10: Score–efficiency comparison of the three planners: (a) total score, (b) planning duration, and (c) planning-token usage. Lower values are better in (b) and (c).

6 Conclusion

Within the task and evaluation setting studied here, the results support the use of LLMs as the high-level closed-loop decision core of a resource-constrained multi-UAV wildfire rescue system. This capability is not uniformly robust: the complete decision loop remains effective in terms of basic task completion, but execution quality and mission continuity deteriorate as operational pressure increases. The proposed LLM-centered closed-loop decision framework addresses this problem by connecting fire-situation understanding and resource-constrained mission planning with Validation, Memory, and Runtime Replanning.

Three sets of evidence support this conclusion. First, the complete framework maintains high basic task completion across four downstream foundation-model configurations, showing that end-to-end operation is not confined to one downstream model, although all configurations lose execution quality under greater operational pressure. Second, the ablations show condition-dependent roles: Validation provides an occasional reliability safeguard, Memory becomes increasingly valuable as resource and scheduling interactions intensify, and Runtime Replanning is critical when perturbations invalidate an active plan. Third, Qwen3-8B fine-tuned with validated planning demonstrations retains 98.24% of the Gemini 3.1 Pro teacher planner's average score on simple and normal episodes, while reducing mean planning time by approximately 63% and planning-token usage by 73%. This final result supports lower-cost local deployment of Mission Planning within the evaluated setting.

This answer remains limited to a calibrated simulation, a fixed episode set, and one real-platform configuration. Moreover, the compact planner has not yet been evaluated under hard and expert runtime conditions. The evaluation also does not include a direct numerical comparison with an external end-to-end rescue method because existing systems assume different inputs, action spaces, resource constraints, and evaluation protocols. The reported evidence therefore supports the framework within the task and evaluation setting defined here. Future work will extend the evaluation to real-flight and mixed real–sim trials, more diverse platforms and fire dynamics, and larger fleets, while developing more selective memory retrieval and more robust, efficient replanning for missions under high operational pressure.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, et al. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning*, pages 287–318, 2022.

Firoj Alam, Hassan Sajjad, Muhammad Imran, and Ferda Ofli. CrisisBench: Benchmarking crisis-related social media datasets for humanitarian information processing. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 15, pages 923–932, 2021. doi: 10.1609/icwsm.v15i1.18115.

James Allan et al. Remote sensing of wildfires: A review of UAV and satellite-based approaches. *Remote Sensing*, 2022.

Andreas Bircher, Mina Kamel, Kostas Alexis, Helen Oleynikova, and Roland Siegwart. Receding horizon "Next-Best-View" planner for 3D exploration. In *Proceedings of the IEEE International Conference on Robotics and Automation*, pages 1462–1468. IEEE, 2016.

David M. J. S. Bowman et al. Human exposure and health impacts of wildfire smoke. *Environmental Research Letters*, 15, 2020.

Anthony Brohan, Noah Brown, Julian Carbajal, Yevgen Chebotar, Chelsea Finn, Karol Hausman, et al. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems*, 2023.

Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, et al. PaLM-E: An embodied multimodal language model. In *Proceedings of the International Conference on Machine Learning*, 2023.

Milan Erdelj, Marija Król, and Enrico Natalizio. Help from the sky: UAV applications in disaster management. *IEEE Transactions on Mobile Computing*, 16(9):2675–2688, 2017.

Dario Floreano and Robert J. Wood. Science, technology and the future of small autonomous drones. *Nature*, 521:460–466, 2015.

Mobin Habibpour, Niloufar Alipour Talemi, John Spodnik, Camren J. Khoury, and Fatemeh Afghah. WildFireVQA: A large-scale radiometric thermal VQA benchmark for aerial wildfire monitoring. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2026.

Xiaoyi Han, Nan Pu, Zunlei Feng, Yijun Bei, Qifei Zhang, Lechao Cheng, and Liang Xue. Benchmarking multi-scene fire and smoke detection. In *Pattern Recognition and Computer Vision: 7th Chinese Conference, PRCV 2024, Urumqi, China, October 18–20, 2024, Proceedings, Part XI*, volume 15041 of *Lecture Notes in Computer Science*, pages 203–218. Springer Nature Singapore, 2024. doi: 10.1007/978-981-97-8795-1_14.

Wenlong Huang, Fei Xia, et al. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning*, 2023.

Alexander Kleiner, Alessandro Farinelli, Sarvapali Ramchurn, Bing Shi, Fabio Maffioletti, and Riccardo Reffato. RMASBench: Benchmarking dynamic multi-agent coordination in urban search and rescue. In *Proceedings of the 12th International Conference on Autonomous Agents and Multiagent Systems*, pages 1195–1196. International Foundation for Autonomous Agents and Multiagent Systems, 2013. Extended abstract.

Jacky Liang, Wenlong Huang, et al. Code as policies: Language model programs for embodied control. In *Proceedings of the IEEE International Conference on Robotics and Automation*, 2023.

Md. Najmul Mowla, Davood Asadi, Kadriye Nur Tekeoglu, Shamsul Masum, and Khaled Rabie. UAVs-FFDB: A high-resolution dataset for advancing forest fire detection and monitoring using unmanned aerial vehicles (UAVs). *Data in Brief*, 55:110706, 2024. doi: 10.1016/j.dib.2024.110706.

M. Popovic et al. Autonomous UAV exploration and mapping for disaster response. *IEEE Robotics and Automation Letters*, 2021.

Dhruv Shah, Michael Equi, et al. LM-Nav: Robotic navigation with large pre-trained models. In *Conference on Robot Learning*, 2023.

Alireza Shamsoshoara, Fatemeh Afghah, Abolfazl Razi, Liming Zheng, Peter Z. Fulé, and Erik Blasch. Aerial imagery pile burn detection using deep learning: The FLAME dataset. *Computer Networks*, 193:108001, 2021. doi: 10.1016/j.comnet.2021.108001.

Sai Vemprala, Rogerio Bonatti, Arthur Bucker, and Ashish Kapoor. ChatGPT for robotics: Design principles and model abilities. *Microsoft Research Technical Report*, 2023.

Sonia Waharte and Niki Trigoni. Supporting search and rescue operations with UAVs. *Survey Literature on UAV-based Disaster Response*, 2021.

Guanzhi Wang, Yu Xie, et al. Voyager: An open-ended embodied agent with large language models. In *Advances in Neural Information Processing Systems*, 2023.

Gui-Song Xia, Xiang Bai, Jian Ding, Zhen Zhu, Serge Belongie, Jiebo Luo, Mihai Datcu, Marcello Pelillo, and Liangpei Zhang. DOTA: A large-scale dataset for object detection in aerial images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3974–3983, 2018. doi: 10.1109/CVPR.2018.00418.

S. Xiao et al. Deep reinforcement learning for UAV autonomous navigation in unknown environments. *IEEE Transactions on Intelligent Transportation Systems*, 2021.

Y. Yang et al. Multi-agent reinforcement learning for cooperative UAV navigation and task allocation. *IEEE Transactions on Neural Networks and Learning Systems*, 2023.

Z. Yang et al. Learning-based UAV 3D reconstruction for autonomous inspection and mapping. *IEEE Robotics and Automation Letters*, 2022.

C. Yu et al. Coverage path planning for UAV-based search and rescue missions. *IEEE Transactions on Automation Science and Engineering*, 2022.

F. Yuan et al. A deep learning framework for fire detection using UAV-based thermal imaging and RGB data. *IEEE Transactions on Geoscience and Remote Sensing*, 2021.

Andy Zeng, Alex Wong, et al. Socratic models: Composing zero-shot multimodal reasoning with language models. In *Proceedings of the International Conference on Learning Representations*, 2023.

Y. Zhang et al. Transformer-based semantic segmentation for UAV remote sensing images. *IEEE Transactions on Geoscience and Remote Sensing*, 2022.

Pengfei Zhu, Longyin Wen, Dawei Du, Xiao Bian, Heng Fan, Qinghua Hu, and Haibin Ling. Detection and tracking meet drones challenge. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11):7380–7399, 2022. doi: 10.1109/TPAMI.2021.3119563.